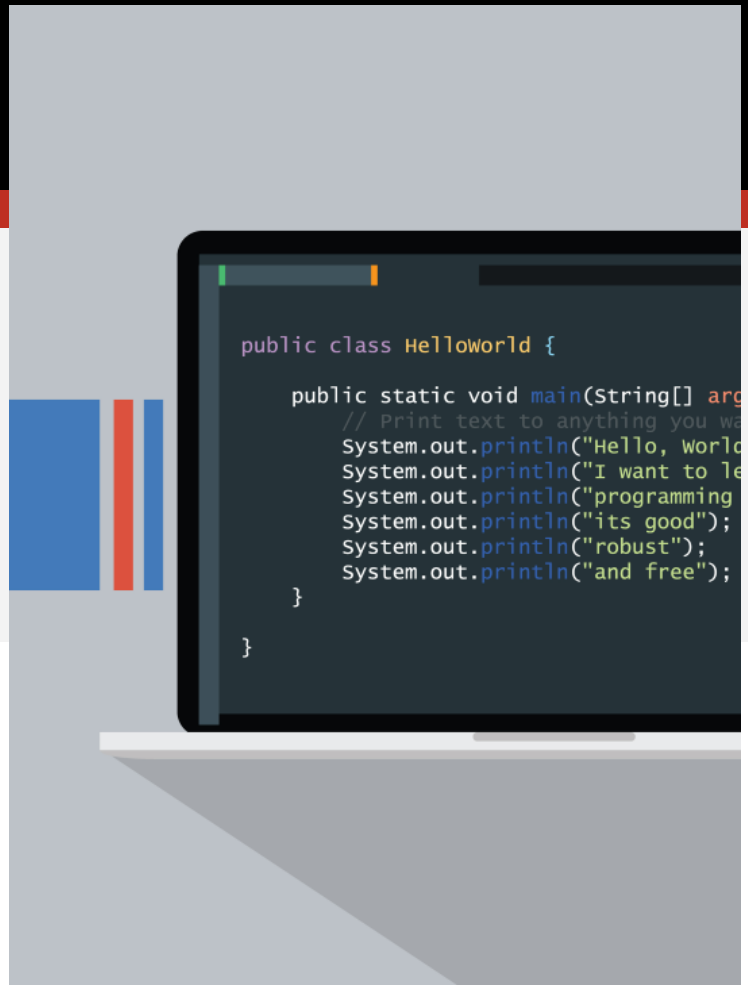
MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



Insertion Sort

Table of Contents

- **Insertion Sort**
- Merge Sort
- Quick Sort & Counting Sort

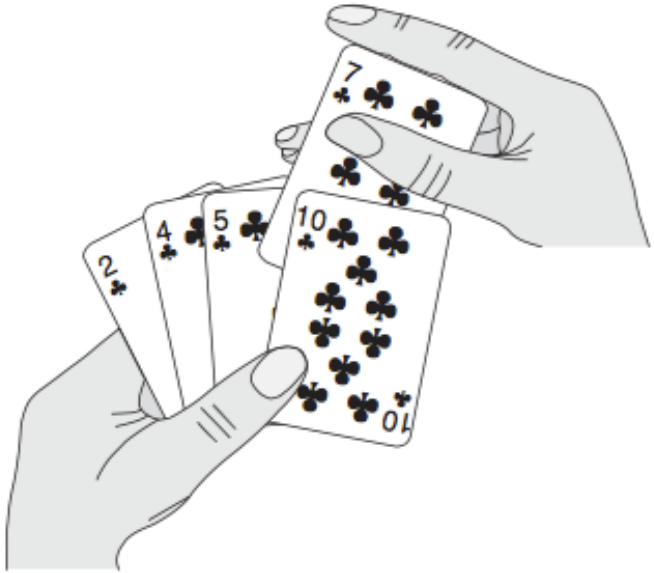
Definitions

- Algorithms: computational procedure to solve problems.
- Problems: Given **a class of input**, demand **output**.
  - Example: Sorting problem
    - **Input:** A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
    - **Output:** A permutation (reordering) $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$
- An input is called an instance of the problem.
  - Example: The input sequence $\langle 31, 41, 59, 26, 41, 58 \rangle$.

Insertion Sort

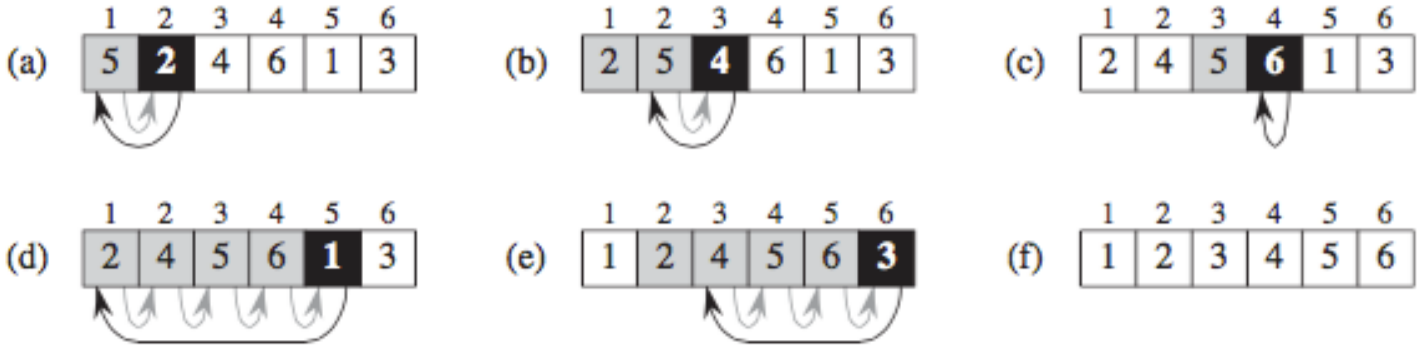
Think about when you play poker, how you arrange the cards?

- Pick a card from the table, where the cards are of random order.
- **Insert** the card to the **appropriate position** of the cards in your hand, which are in the correct order.



Example Walkthrough

- Insertion sort works in the same way: insert a new card into a sorted subarray.



Pseudocode

- Pseudocode is for humans to understand, while code is for computers to understand.
- Conventions:
  - Indentation indicates block structure.
  - The symbol "//" means comment.
  - Parameters are **passed by value, not passed by reference**.

```
INSERTION-SORT(A):
    for j = 2 to A.length:
        key = A[j]
          // insert A[j] into the sorted sequence A[1..j-1]
        i = j - 1
        while i > 0 and A[i] > key:
            A[i + 1] = A[i]
            i = i - 1
        A[i + 1] = key
```

Time Complexity

- **Complexity** refers to how efficiently an algorithm or data structure performs with respect to:
  - **Time** (how fast it runs)
  - **Space** (how much memory it uses)
- **Time complexity** of an algorithm means the number of instructions need to finish it.
- In general, the time taken by an algorithm grows with the size of the input.
  - number: # of bits to represent the number
  - array: length
  - graph: # of vertices and edges

Complexity Analysis (1 of 3)

- Let t_j be the number of times the while loop test in line 5 is executed for the value of j.

INSERTION-SORT(A):	cost	times
1 for j = 2 to A.length:	c_1	n
2 key = A[j]	c_2	n-1
3 // insert A[j] into the		
3 sorted sequence A[1..j-1]		
4 i = j - 1	c_4	n-1
5 while i > 0 and A[i] > key:	c_5	n-1
6 A[i + 1] = A[i]	c_6	$\sum_{j=2}^{n} t_j$
7 i = i - 1	c_7	$\sum_{j=2}^{n}(t_j - 1)$
8 A[i + 1] = key	c_8	n-1

- Sum the cost, and get

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1)$$

Complexity Analysis (2 of 3)

- When the array is sorted, $t_j = 1$ for $j = 2,3, \dots, n$.
  - We get the **best-case** running time

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1)$$
$$= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8).$$

- When the array is reverse sorted, $t_j = j$ for $j = 2,3, \dots, n$.
  - We get the **worst-case** running time

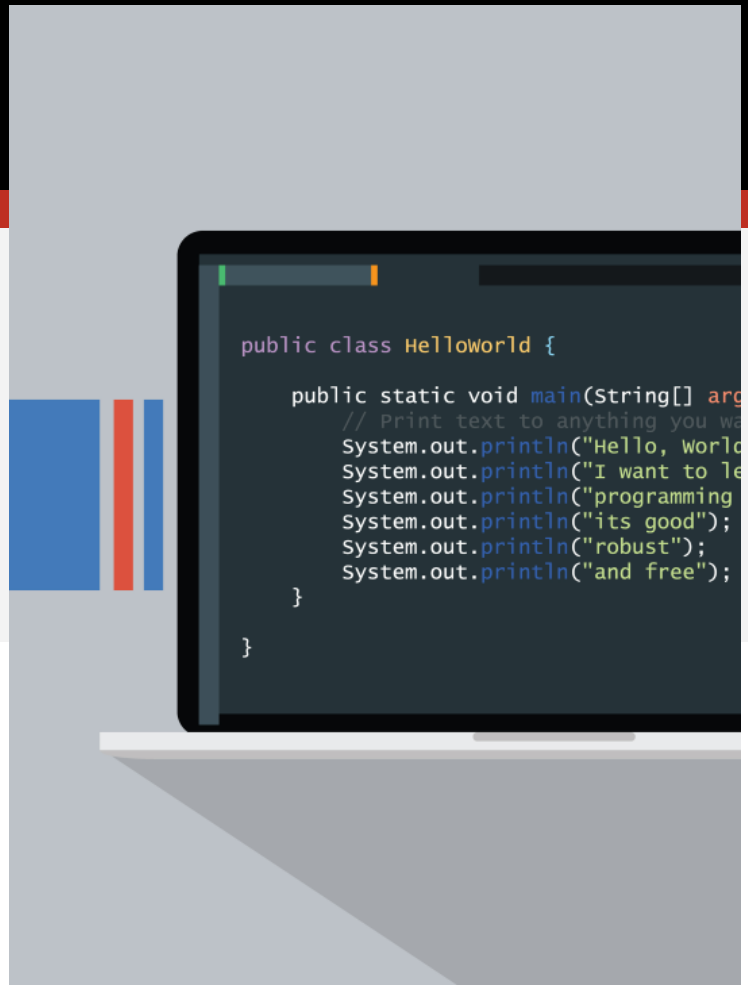
$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right)$$
$$+ c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1)$$
$$= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8\right)n$$
$$- (c_2 + c_4 + c_5 + c_8).$$

Complexity Analysis (3 of 3)

- We are more interested in the rate of growth of the running time.
  - We only consider the leading term of a formula (e.g., an^2).
  - We ignore the leading term's constant coefficient.
- We are more interested in the worst-case complexity.
  - **Worst-case complexity**: $O(n^2)$.
  - **Average-case complexity**: $\Theta(f(n))$.
- We say one algorithm is more efficient than another if the worst-case complexity has a lower order of growth.

MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



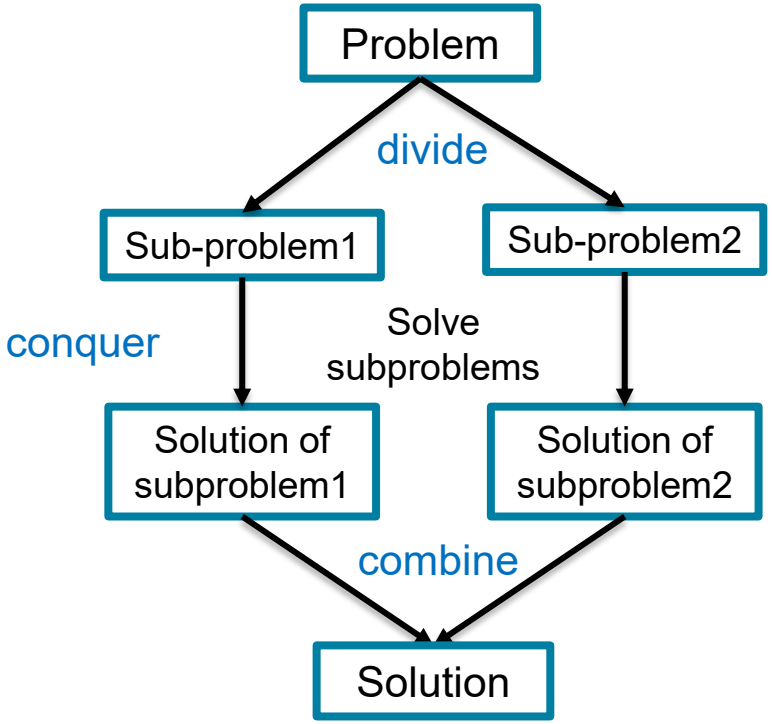
Merge Sort

Table of Contents

- Insertion Sort
- **Merge Sort**
- Quick Sort & Counting Sort

Divide and Conquer

- **Divide-and-conquer** paradigm:
  - Break the problem into several subproblems that are similar to the original one but smaller in size.
  - Solve the subproblems recursively.
  - Combine these solutions to create a solution to the original problem.



Divide and Conquer

- Each level of recursion involves the three steps:
  - **Divide**: the problem into a number of subproblems that are smaller instances of the same problem.
  - **Conquer**: the subproblems by solving them recursively. If the subproblem sizes are small enough, however, just solve the subproblems in a straightforward manner.
  - **Combine**: the solutions to the subproblems into the solution for the original problem.

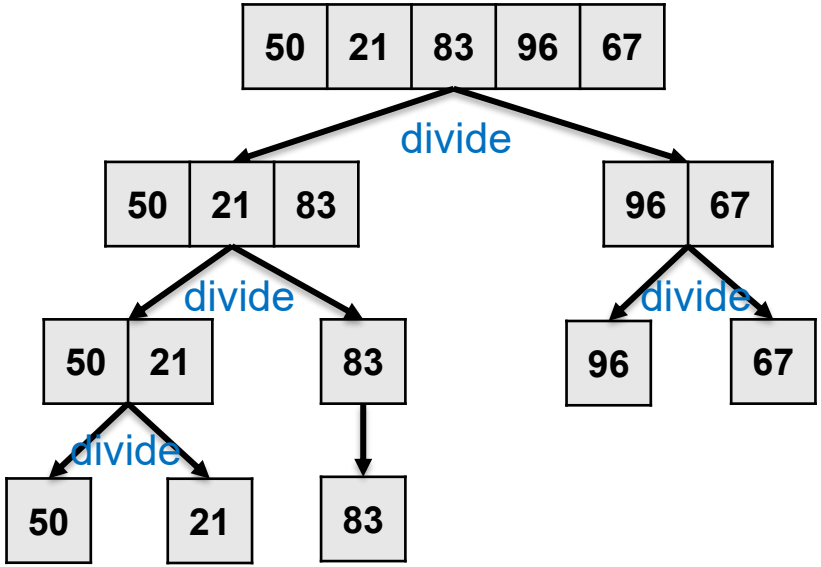
Merge Sort

- Merge sort applies the logic of divide-and-conquer.
  - **Divide**: Divide the n-element sequence to be sorted into subsequences of n/2 elements each.
  - **Conquer**: Sort the two subsequences recursively using merge sort.
  - **Combine**: Merge the two sorted subsequences to produce the sorted answer.
- We recursively divide the sequence until it reaches the length of 1, as it must be sorted.
- The key operation is to combine two sorted subsequences into one.

Example Walkthrough (1 of 2)

Division Process:

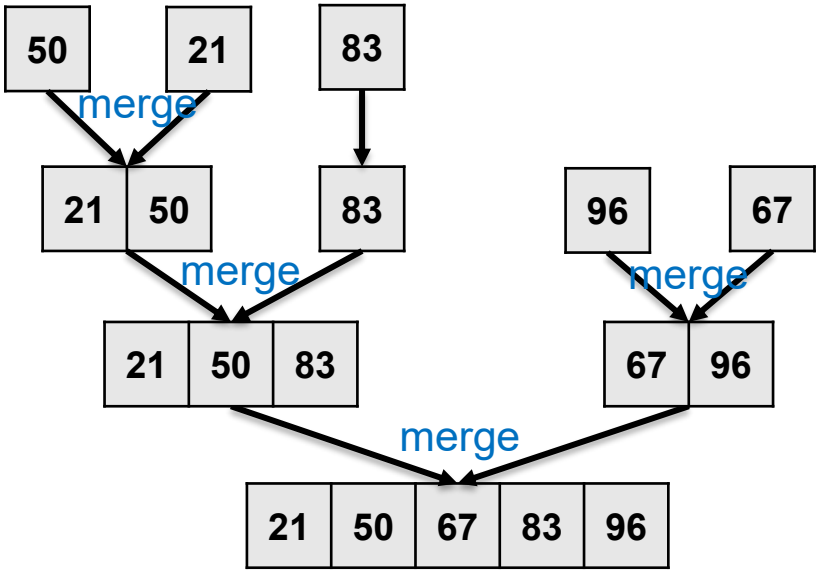
- Recursively divide the array into subarrays, until each subarray can be divided no further.



Example Walkthrough (1 of 2)

Merge Process:

- Since all subarrays now contain one element each, the merge process will start.
- Subarrays $\langle 50 \rangle$ and $\langle 21 \rangle$ will merge in sorted order to form subarray $\langle 21, 50 \rangle$. Finally, two subarrays $\langle 21, 50, 83 \rangle$ and $\langle 67, 96 \rangle$ will merge to produce the final sorted array.

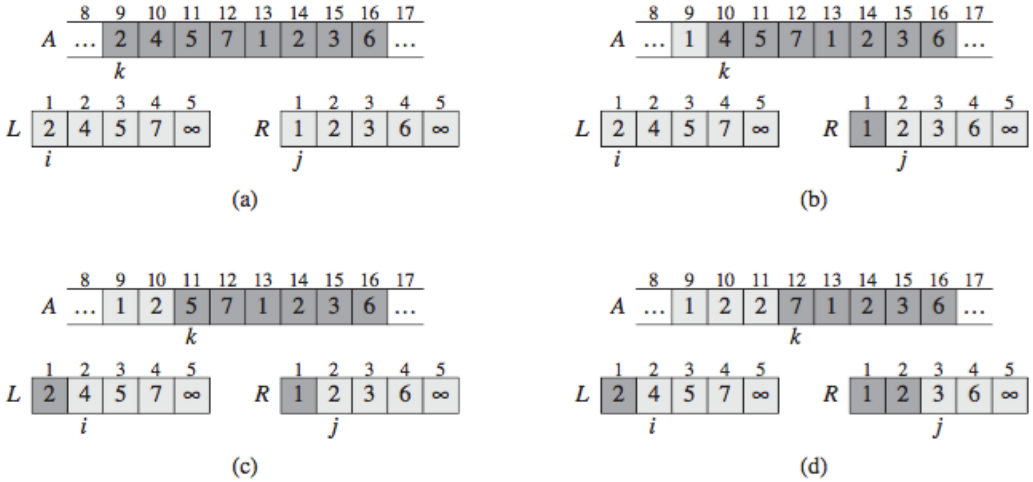


Pseudocode: Merge

```
MERGE(A, p, q, r)
n1 = q - p + 1
n2 = r - q
let L[1 .. n1 + 1] and R[1 .. n2 + 1] be new arrays
for i = 1 to n1
    L[i] = A[p + i - 1]
for j = 1 to n2
    R[j] = A[q + j]
L[n1 + 1] = ∞
R[n2 + 1] = ∞
i = 1
j = 1
for k = p to r
    if L[i] ≤ R[j]
        A[k] = L[i]
        i = i + 1
    else A[k] = R[j]
        j = j + 1
```

copy them to L and R

iteratively copy the smaller one into the new array



Pseudocode: Merge Sort

- Iteratively split the array into two subarrays, until the array's size is 1.
- Merge the subarrays.

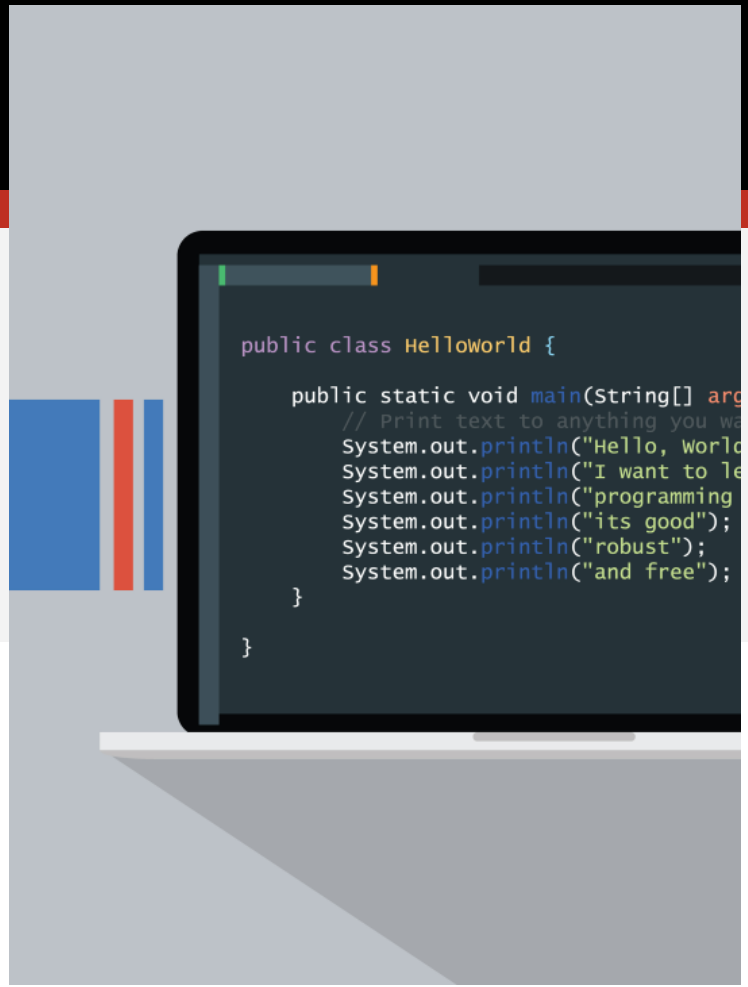
```
MERGE-SORT(A,p,r)
    if p < r
        q = ((p+r)/2)
        MERGE-SORT(A,p,q)
        MERGE-SORT(A,q + 1,r)
        MERGE(A,p,q,r)
```

Complexity Analysis

- The time complexity of merge sort is $O(nlogn)$.

MONTCLAIR
STATE UNIVERSITY

CSITBG2 – Data Structures and Algorithms: Java



Quick Sort & Counting Sort

Table of Contents

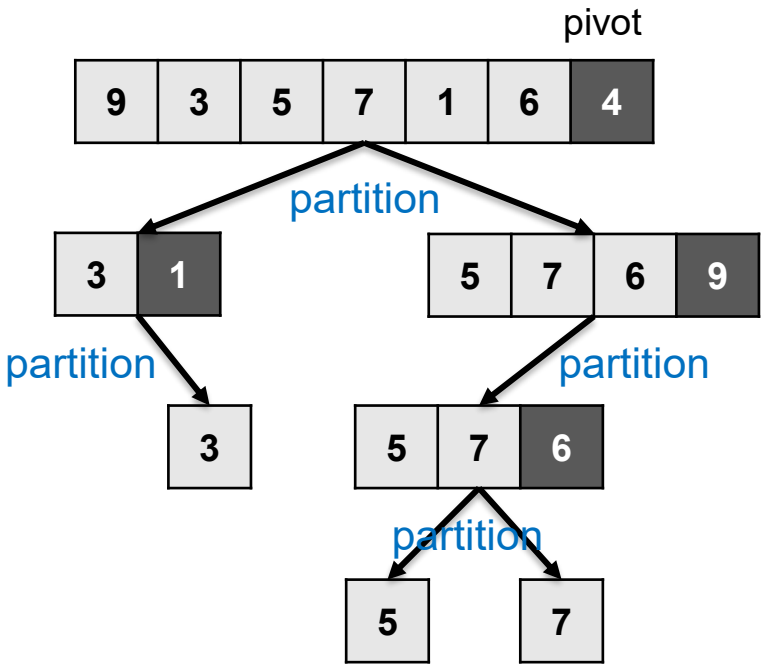
- Insertion Sort
- Merge Sort
- **Quick Sort & Counting Sort**

Quick Sort

- We apply the divide-and-conquer strategy in Quick Sort, too.
  - **Divide:** Partition (rearrange) the array A[p..r] into two (possibly empty) subarrays A[p..q-1] and A[q+1..r] such that each element of A[p..q-1] is less than or equal to A[q], which is, in turn, less than or equal to each element of A[q+1..r]. Compute the index q as part of this partitioning procedure.
  - **Conquer:** Sort the two subarrays A[p..q-1] and A[q+1..r] by recursive calls to quicksort.
  - **Combine:** Because the subarrays are already sorted, no work is needed to combine them: the entire array A[p..r] is now sorted.
- But most of the work are in the Divide step.

Example Walkthrough

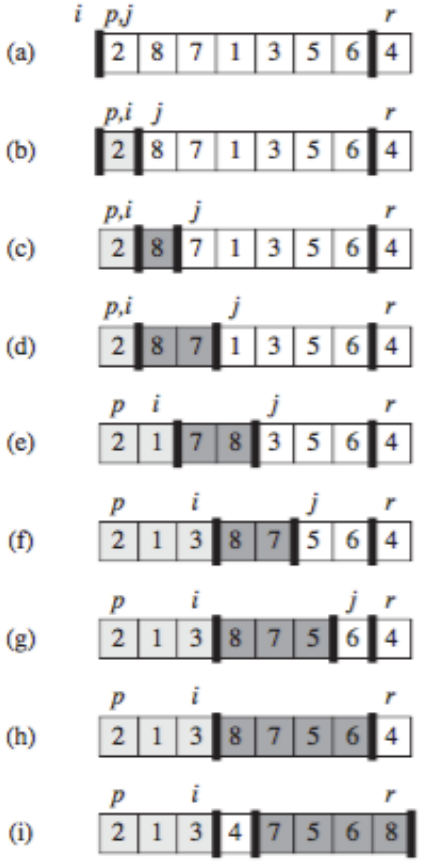
- **Choose a Pivot**: Select an element as the pivot (commonly the last, first, or a random element).
- **Partition the Array**: Rearrange elements so that:
  - Elements **smaller** than the pivot move to its **left**.
  - Elements **greater** than the pivot move to its **right**.
- **Recursively Apply Quick Sort**:
  - Sort the left subarray.
  - Sort the right subarray.
  - **Base Case**: If the subarray has one or zero elements, it is already sorted.



Pseudocode

```
PARTITION(A, p, r)
    x = A[r]
    i = p - 1
    for j = p to r - 1
        if A[j] <= x
            i = i + 1
            exchange A[i] with A[j]
    exchange A[i + 1] with A[r]
    return i + 1

QUICKSORT(A,p,r)
    if p < r
        q = PARTITION(A, p, r)
        QUICKSORT(A, p, q - 1)
        QUICKSORT(A, q + 1, r)
```



Complexity Analysis

- Complexity: $O(n^2)$
- So far, we have learned sorting algorithm with $O(nlogn)$ complexity: Merge Sort; and two algorithms with $O(n^2)$ complexity.
- But in practice, Quick Sort is the most efficient one.

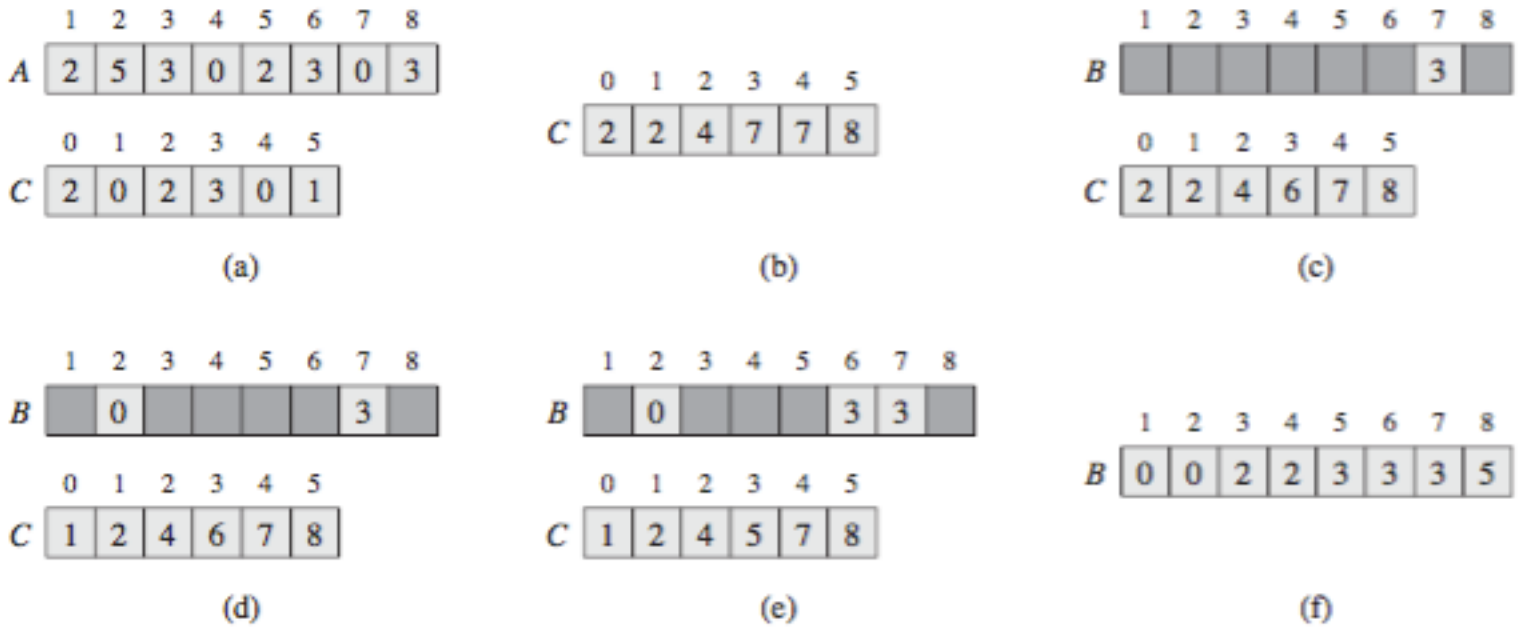
Counting Sort

- Is it possible that we get any sorting algorithm with complexity better than $O(nlogn)$?
- Yes, but it comes with condition: each of the n input elements is an integer in the range 0 to k, for some integer k. When k=$O(n)$, the sort runs in $O(n)$ time.

Pseudocode

```
COUNTING-SORT(A, B, k)
let C[0..k] be a new array
for i = 0 to k
    C[i] = 0
for j = 1 to A.length
    C[A[j]] = C[A[j]] + 1
// C[i] now contains the number of elements equal to i.
for i = 1 to k
    C[i] = C[i] + C[i − 1]
// C[i] now contains the number of elements less than or equal to i.
for j = A.length downto 1
    B[C[A[j]]] = A[j]
    C[A[j]] = C[A[j]] − 1
```

Example Walkthrough



Complexity Analysis

- Complexity: $O(k + n)$

MONTCLAIR
STATE UNIVERSITY